Optimising Supply Chain Logistics Using Algorithm Development and Mathematical Concepts

Scenario

As a data analyst at a logistics company dedicated to refining its supply chain operations, your mission is to craft algorithms and make use of mathematical principles to enhance operational efficiency.

An overview of supply chain logistics, including the elements of the supply chain and how they interact as a system, is provided at the end of this assignment brief. Please read this overview carefully to understand supply chain management before starting your report.

Write a report to explore several essential factors vital for optimising supply chain logistics through algorithm development and mathematical concepts.

Task 1: Optimal Route Planning and Propositional Logic for Supply Chain Logistics.

This section illustrates the application of Optimal Route Planning and Propositional Logic for Supply Chain Logistics to increase operational efficiency and decision making.

Proposition in a definition is a statement that is either true or false. According to Oxford Department of Computer Science (Lec07, P04), “A proposition is a statement that can be either true or false; it must be one or the other, and it cannot be both.” In supply chain logistics, utilizing propositional logic can help in modelling decision making processes such as route planning.

For example, Propositions R,S,T,U are used for planning a route. And each proposition represent a particular decision making process,

R= Vehicle is ready for transportation

S= Road conditions are suitable for travel

T= Product must be delivered within 24 hours

U= Weather conditions are suitable for travel

Each of these propositions will be evaluated whether they are true or false based on real life conditions.

A Pseudocode will be created for an algorithm that selects the optimal routes based on the truth values of propositions. Psuedocode will demonstrate how propositional logic can be used to create an algorithm for selecting the suitable route.

```
If ( R is true and S is true and T is true and U is true ) then select_optimal_route();
else
```

choose_alternative_route(); end if

This code will evaluate if all values are meeting the requirements. If one of which appears to be false, then the code will automatically check for an alternative route.

Propositional logic is an efficient methodology for dealing and handling simple decision making processes in terms of supply chain logistics by using simple statements.These statements could be either true or false and cannot be both. However, it cannot handle complex decision making scenarios or uncertainties because " It only handles binary truth values (true/false) and cannot represent probabilities." (geeksforgeeks.org, 25 apr, 2025). For example, there is a 60% chance that it will rain the day the delivery will take place. Hence, the probability of delaying the delivery could be high. Propositional logic cannot deal with such situations. Instead, "One way to come up with approximate answers to a problem is to use a heuristic, a technique that guides an algorithm to find good choices. When an algorithm uses a heuristic, it no longer needs to exhaustively search every possible solution, so it can find approximate solutions more quickly. A heuristic is a shortcut that sacrifices accuracy and completeness." (khanacademy.org).

Summary:

Overall, this section has covered the implementation of propositional logic into the supply chain logistics and how it helps in modelling simple decision making processes such as route planinng.

A pseudocode was created to demonstrate how propositional logic could be used to create basic algorithms for planning a route.

An alternative methodology was suggested for an efficient solution in terms of dealing with complex decision making processes and uncertanties such as proposing approximate answers for a problem. Heuristic was the favorable methodology because it searches for a solution quickly despites the accuracy of the presented solution. Favoring a quicker solution over accuracy.

Task 2: Demand Forecasting and Predicate Logic.

This section focuses on utilizing demand forecasting and predicate logic to predict future product demands.

“Predicates and Quantifiers are fundamental concepts in mathematical logic, essential for expressing statements and reasoning about the properties of objects within a domain.” (geeksforgeeks.org , 24 apr, 2025).

A Predicate is a sentence depending on variables which becomes a statement upon substituting values for those variables within a domain.

Since it includes variables, a predicate can deal and perform complex relationships. Unlike propositions. In the upcoming example, Predicates will be representing factors influencing demand forecasting in supply chain logistics. Such as market conditions, competitor actions, and promotional strategies.

Predicate M(x)= Market conditions affect the demand for product x.

Predicate C(x)= Competitor actions affect the demand for product x.

Predicate P(x)= Promotional strategies affect the demand for a product x

Predicate H(x)= Historical sales shows the future demand for a product x

While predicates help in creating statements, quantifiers specify these statements by identifying how many elements within a domain are true. There are two types of Quantifiers which are (Universal Quantifiers) and (Existential Quantifiers). Quantifiers will be used in making assertions about the impact of the created predicates on overall demand trends in the following example.

The Universal Quantifier means (for all or for every). It is symbolized as ($\forall$)

The Existential Quantifier means (there exists). It is also symbolized as ($\exists$)

After identifying the predicates, Quantifiers will be applied to make general statements about these predicates.

When using Universal Quantifier, the statement would appear as:

(for all products x, if market conditions, competitor actions, promotional strategies, and Historical sales affect positively the demand for a product. Then the demand for product x is likely to increase.)

The logical form of universal quantifier would appear as:

$\forall \mathbf{x}(\mathbf{M}(\mathbf{x}) \wedge \mathbf{C}(\mathbf{x}) \wedge \mathbf{P}(\mathbf{x}) \wedge \mathbf{H}(\mathbf{x}) \rightarrow \mathbf{forecast_increase}(\mathbf{x}))$.

When using Existential Quantifier:

The statement= there exists product x such that market conditions, competitor actions, promotional strategies, and historical sales that positively affect the demand for product x .

The logical form= $\exists \boldsymbol{x}(\mathbf{M}(\boldsymbol{x}) \wedge \mathbf{C}(\boldsymbol{x}) \wedge \mathbf{P}(\boldsymbol{x}) \wedge \mathbf{H}(\boldsymbol{x}) \rightarrow \mathbf{forecast_increase}(\boldsymbol{x}))$

By combining Universal and Existential Quantifiers, we will get:

$\forall x(M(x) \rightarrow \exists y(P(y) \wedge forecast_increase(x)))$

This means that for every product x that is positively affected by market conditions, there exists some promotional strategy y that will potentially increase the demand for product x.

Applying Quantifiers to The statements will be validated by applying the data provided:

1- to predict future demand for the product, rules and impacts will be created and combined with predicates.

impacts:

1- If market conditions are good for selling sports goods, increase demand by 10%.

2- If competitor actions are weak for achieving better sales in sports goods , increase demand by 15%.

First rule:

$forecast_increase_market(x)=current_demand(x)\times 1.10$

Second rule:

$forecast_increase_competitor(x)=current_demand(x)\times 1.15$

The current demand for product x1=150 units.

m(x1)= true, because market conditions are good

c(x1)= false, because competitor actions are not weak.

By applying the rule:

150×1.10=165 units

Second rule for c(x1) is false, so no increase will occur. Final demand= 165

The current demand for product x2= 80 units

m(x2)= true, because market conditions are good.

c(x2)= true, because competitor actions are weak.

By applying the rule:

First rule= 80×1.10=88 units

Second rule= 88×1.15=101.2 units. Final demand= 101.2 units.

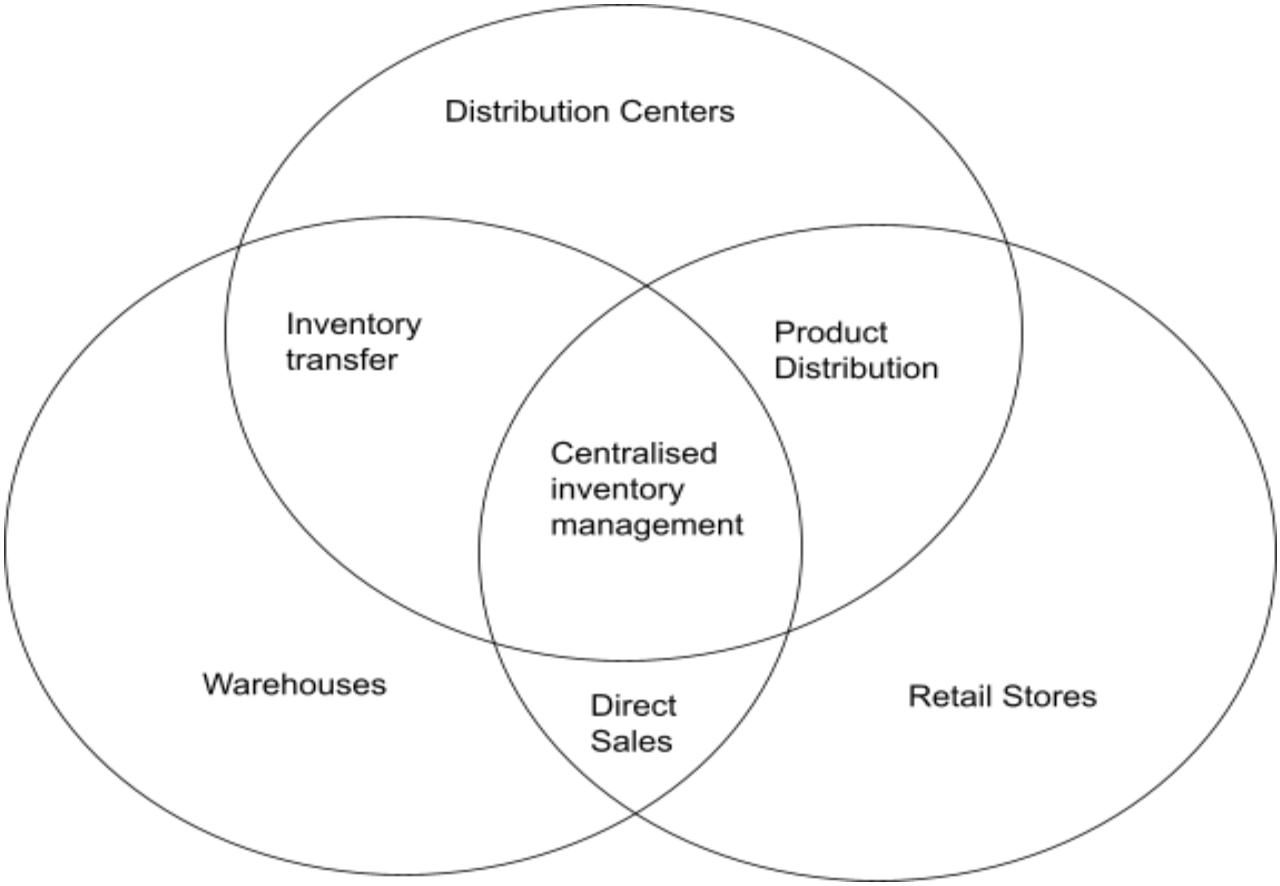
Summary:

Overall, this section explored the application of quantifiers and predicates into the supply chain logistics and how it can affect the supply chain. By creating predicates, applying quantifiers, and implementing rules to validate statements. Modeling can be applied to easily predict future demand for products.

Task 3: Visualisation Techniques and Selection of Efficient Data Structures.

This task focuses on implementing algorithms for supply chain and utilizing venn diagrams to visually represent relationships between elements in the supply chain logistics network.

The following diagram is venn diagram that illustrates the relationships between different elements in the supply chain logistics network:



The diagram represents the relationships between three major elements in the supply chain.

Warehouses : it is a place where the inventory is being stored and managed, the diagram illustrates the significant relationship of warehouses and distribution centers, where inventory is being transferred to be ready for the distribution.

Distribution centers: it is the place where the inventory that was transferred from warehouses is being prepared to be ready for the distribution.

Retail stores: the place where products get displayed for the customers in order to get sold. It has a strong relationship with the distribution centers since products get categorized for particular retail stores before the final distribution.

Direct sales: this section explains the action of products getting sold directly to customers as explained.

These relationships between elements are essential for every supply chain company as it optimizes its operations and ensures a flow control of inventory. Leading to reducing the costs of product distribution and meeting the customer demands and satisfactions.

Key	Value
Warehouses	Location of products storage ahead of distribution
Distribution Centers	Hubs for product sorting and shipping
Retail Stores	Locations where consumers purchase things
Inventory Management	A system to monitor product movement and stock levels
Route planning	The process of identifying the most effective routes for goods

The provided table is a hash table that shows how data is being stored and processed in the supply chain logistics to ensure data efficiency.

The table illustrates the two main elements of hashing. By applying the hash data structure, all data needed for a particular element within a table can be found easily using the key. It is like creating a table of contents for a book. Instead of searching the entire book for specific information, a table of content could be used in order to navigate the desired information easily.

After creating a hash table to store and process data for different elements in the supply chain, an algorithm will be created to work with the data structure selected (Hashing) to perform a specific task.

```
function updateInventory(key, quantity):
if key exists in hashTable:
if quantity is positive:
// New stock has arrived
hashTable[key] += quantity
```

```
else if quantity is negative:

// Stock has been sold

hashTable[key] += quantity

if hashTable[key] < 0:

print("Error: Not enough inventory")

// Handle the error, maybe by setting the inventory to 0

hashTable[key] = 0

Else:

print("Error: Item not found in inventory")
```

Once a new stock for a specific product arrives or gets sold, the above algorithm will help in managing and updating the inventory levels in warehouses constantly.

```
function findBestRoute(start, end):

// This function will suppose that we have a graph that shows the best routes available with distances  between distribution centers and retail stores.

 routes = getRoutesFromGraph(start, end)

bestRoute = None

minDistance = Infinity

For a route in routes:

distance = calculateDistance(route)

if distance < minDistance:

minDistance = distance
```

```
bestRoute = route

return bestRoute
```

The algorithm provided above helps in finding the best possible route for the distribution of the product.

Summary:

Overall, this section illustrates how Visualization techniques, and selecting a specific data structure for the supply chain logistics tailored for a specific company could optimize the company's supply chain logistics operations . A venn diagram was created to show the relationships between elements in the supply chain. Additionally, data structure was selected to ensure efficient processing and storing of data. Finally, an algorithm created to work efficiently with the selected data structure to perform a specific task.

Task 4: Algorithm Comparison for Supply Chain Logistics.

This section explores the algorithms, the types of algorithms, a comparison between the types of algorithms, and what best suits the supply chain logistics for a company.

> An algorithm is a process where computational problems get solved using particular instructions tailored for that problem. techtarget.com claims that An algorithm is a procedure used for solving a problem or performing a computation. Algorithms act as an exact list of instructions that conduct specified actions step by step in either hardware- or software-based routines. An algorithm could be used for sorting sets of numbers or for more complicated tasks, such as recommending user content on social media.

In the supply chain logistics, algorithms were used to navigate the data desired easily. However, in order to find, process, and sort data clearly, it requires particular algorithms to do such a job. This involves (searching and sorting algorithms) to ensure efficient data access.

According to geeksforgeeks.org (2024),"A **Sorting Algorithm** is used to rearrange a given array or list of elements in an order. For example, a given array [10, 20, 5, 2] becomes [2, 5, 10, 20] after sorting in increasing order and becomes [20, 10, 5, 2] after sorting in decreasing order".

Sorting algorithms involve Quicksort and MergeSort algorithms.

QuickSort is a sorting method which is based on the Divide and Conquer strategy, selects an element as a pivot and divides the provided array around the pivot by positioning it correctly within the sorted array. (geeksforgeeks.org , 17, apr, 2025)

Advantages of QuickSort algorithm:

1- it works effectively with large data sets

Disadvantages of QuickSort algorithm:

1- When the pivot is badly chosen, it has a worst-case time complexity of O(n2).

MergeSort : this algorithm follows the divide and conquer approach as QuickSort, it is also known for its stability and efficiency. (geeksforgeeks.org , 25 apr, 2025)

Advantages of MergeSort algorithm:

1- Because merge sort is a stable sorting method, the relative order of equal elements in the input array is maintained.

Disadvantages of MergeSort algorithm:

1- Since QuickSort operates in-place and is more cache-friendly, Merge Sort is generally slower than QuickSort. (geeksforgeeks.org , 25 apr, 2025)

On the other hand, Searching algorithms in general are “essential tools in computer science used to locate specific items within a collection of data.” (geeksforgeeks.org , 13 apr, 2025)

There are two most common types of algorithms that are usually used in searching algorithms:

Linear search algorithm: It is applied to an array that is not ordered. It mostly compares each item to be searched with array items one at a time. It requires O(n) time, or linear time.

Binary search algorithm: It applies to a sorted array. The array's middle member is mostly compared first, and if it matches the input, it returns. (geeksforgeeks.org , 13 apr, 2025)

Advantages of linear search:

1- this algorithm is usually best suited for small data sets

Disadvantages of linear search:

1- For large datasets, linear search is slow due to its O(N) time complexity. (geeksforgeeks.org ,27 mar, 2025)

Advantages of binary search:

1- Because the amount of data that needs to be searched is halved with each step, it is significantly faster than a linear search. (bbc.co.uk)

Disadvantages of binary search:

1-Binary search is limited to sorted lists or arrays. An extra O(n log n) time complexity is introduced if the data is unsorted since it must first be sorted. (youcademy.org)

When dealing with large datasets for supply chain logistics, generally, companies in this case are often looking to do something with that dataset in real time. That being said:

QuickSort is preferred as it is considered the most efficient sorting on average case and it is in place sorting (regardless of the potential worst case use case)

MergeSort is preferred due to its stability, efficient space consideration and is just a little less preferred than QuickSort since it is not in place sorting by virtue of the required space.

Linear Search is not acceptable due to the speed with large datasets.

Binary Search is quite efficient with large datasets, although the limitation here is that they need to be sorted first before searching.

Summary:

Typically, QuickSort and Binary Search are the most efficient types of algorithms for large datasets in supply chain logistics, if the data is sorted appropriately for Binary Search.

MergeSort, though, has stability and consistent performance and is a good option at the cost of space.

A Linear Search is not great when the dataset is small or it could be worse, when the data cannot be sorted.

Overall, this section has explored different types of algorithms and their functions and how they can be applied to the supply chain to optimize various aspects of its logistics, ensuring better sorting of large data sets to manage the inventory smoothly.

References:

https://www.cs.ox.ac.uk/people/michael.wooldridge/teaching/soft-eng/lect07.pdf

https://www.geeksforgeeks.org/propositional-logic-in-artificial-intelligence/

https://www.khanacademy.org/computing/ap-computer-science-principles/algorithms-101/solving-hard-problems/a/using-heuristics

https://www.geeksforgeeks.org/mathematic-logic-predicates-quantifiers/

https://www.techtarget.com/whatis/definition/algorithm

https://www.geeksforgeeks.org/sorting-algorithms/

https://www.geeksforgeeks.org/quick-sort-algorithm/

https://www.geeksforgeeks.org/searching-algorithms/

https://www.geeksforgeeks.org/linear-search/https://www.bbc.co.uk/bitesize/guides/zgr2mp3/revision/4#:~:text=One%20of%20the%20main%20advantages%20of%20a%20binary%20search%20close,searched%20halves%20with%20each%20step.

https://youcademy.org/pros-cons-of-binary-search-algorithm/